

Scope :

Scope determines the accessibility of variables and functions at different parts of the code. There are three main types of scope: global scope, function scope, and block scope.

1. Global Scope

Variables declared outside any function or block have global scope. They are accessible from anywhere in the code.

```
var globalVar = 'I am global';

function myFunction() {
  console.log(globalVar); // Accessible here
}

myFunction();
console.log(globalVar); // Accessible here
```

2. Function scope

Variables declared within a function are in function scope. They are accessible only within that function.

```
function showAge() {
  let age = 25;
  console.log(age); // Accessible inside function
}

showAge();
// console.log(age); // Error: age is not defined
```

3. Block scope

Variables declared with `let` or `const` within a block (e.g., within curly braces `{}`) have block scope. They are accessible only within that block.

```
if (true) {  
  let blockVar = "I am block scoped";  
  const anotherVar = 10;  
  console.log(blockVar); // Accessible here  
}  
  
// console.log(blockVar); // Error: blockVar is not defined
```

4. Lexical scope

JavaScript uses lexical scoping, meaning that the scope of a variable is determined by its location within the source code. Nested functions have access to variables declared in their outer scope.

```
function outerFunction() {  
  var outerVar = 'I am outside';  
  
  function innerFunction() {  
    console.log(outerVar); // Accessible here  
  }  
  
  innerFunction();  
}  
  
outerFunction();
```

Events:

In JavaScript, events are actions or occurrences in the browser that JavaScript can respond to. They usually happen due to user interactions or browser activities.

Essential Elements of JavaScript Events

- **Trigger:** Events are triggered by user actions (like clicking a button, typing into a form field, or moving the mouse), or by the browser itself (like a page finishing loading, or an image failing to load).
- **Event Handler:** A **function** that is executed when a specific event occurs. This function contains the code that responds to the event.
- **Event Object:** When an event occurs, the browser creates an **Event Object** which contains important details about the event (e.g., which key was pressed, where the mouse was clicked, etc.) and is passed as an argument to the event handler function.

Types of Events:

1. Mouse Events

Triggered by mouse actions.

Event	Description
click	Triggered when an element is clicked
dblclick	Triggered when an element is double-clicked
mousedown	Triggered when a mouse button is pressed down

mouseup	Triggered when a mouse button is released
mouseenter	Triggered when the mouse pointer enters an element
mouseleave	Triggered when the mouse pointer leaves an element
mousemove	Triggered when the mouse pointer moves over an element
mouseover	Triggered when the mouse pointer moves over an element or its children
mouseout	Triggered when the mouse pointer moves out of an element or its children
contextmenu	Triggered when the right mouse button is clicked (to open context menu)
wheel	Triggered when the mouse wheel is scrolled

2. Keyboard Events

Triggered by keyboard actions.

Event	Description
keydown	Key is pressed down
keyup	Key is released
keypress	Key is pressed (deprecated; use `keydown`)

3. Form Events

Triggered by interaction with form elements.

Event	Description
submit	Triggered when a form is submitted
change	Triggered when the value of an input, select, or textarea changes
focus	Triggered when an input or textarea gains focus

blur	Triggered when an input or textarea loses focus
input	Triggered immediately when the value of an input or textarea changes

4. Window Events

Triggered by changes to the browser window.

Event	Description
load	Page has fully loaded
resize	Browser window is resized
scroll	User scrolls the page

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Events Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 40px;
    }

    #mouseBox {
      width: 200px;
      height: 100px;
      background-color: lightblue;
      line-height: 100px;
      margin: 20px auto;
      cursor: pointer;
      border-radius: 8px;
    }

    #output {
      margin-top: 20px;
      font-weight: bold;
      color: darkblue;
    }

    input {
      padding: 8px;
      width: 200px;
      margin: 10px;
    }

    form {
      margin-top: 30px;
    }
  </style>
</head>
<body>
  <h1>JavaScript Events Demo</h1>

  <!-- Mouse Event -->
  <div id="mouseBox">Click Me</div>
```

```

<!-- Keyboard Event -->
<input type="text" id="keyInput" placeholder="Type something..." />

<!-- Form Event -->
<form id="myForm">
  <input type="text" id="nameInput" placeholder="Enter your name" required />
  <button type="submit">Submit</button>
</form>

<p id="output">Try interacting with the elements!</p>
<script src="script.js"></script>
</body>
</html>

```

[script.js](#)

```

// Mouse Event: click
document.getElementById("mouseBox").addEventListener("click", function() {
  this.style.backgroundColor = "lightgreen";
  document.getElementById("output").textContent = "Mouse Click Event Triggered!";
});

// Keyboard Event: keydown
document.getElementById("keyInput").addEventListener("keydown", function(event) {
  document.getElementById("output").textContent = `Key pressed: ${event.key}`;
});

// Form Event: submit
document.getElementById("myForm").addEventListener("submit", function(event) {
  event.preventDefault(); // Prevents page reload
  const name = document.getElementById("nameInput").value;
  document.getElementById("output").textContent = `Form submitted! Hello, ${name}.`;
});

// Window Event: resize
window.addEventListener("resize", function() {
  document.getElementById("output").textContent = "Window resized!";
});

```


String:

A String in JavaScript is a primitive data type that represents a sequence of characters enclosed in single, double, or backticks, used to store and manipulate text data, and it is immutable in nature.

```
let str1 = 'Hello';
console.log(str1);

let str2 = "World";
console.log(str2);

let str3 = "Alice";
let greeting = `Hello, ${str3}!`; // String Interpolation
console.log(greeting);
```

1. Immutability:

```
//Immutable
let str = "Hello";
str[0] = "J";
console.log(str); // "Hello" (unchanged)
```

```
let str = "Hello";
str = "World";

console.log(str);
```

Explanation :

1. "Hello" is created in memory.
2. str points to "Hello".
3. When you assign "World", a new memory location for "World" is created.
4. The variable str now points to the new location.
5. The old "Hello" string remains in memory until garbage collection removes it (if no variable references it).

2. String Intern :

When you assign a string variable, it holds a **reference** (pointer) to the string value.

String interning means that **identical string values are stored only once in memory**, and all variables that have the same literal value **share the same memory reference**.

```
let s1 = "hello";
let s2 = "hello";
console.log(s1 === s2); //  true → likely same interned string
```

3. String Concatenation :

```
let a = "Hello";
let b = "World";
let c = a + " " + b;

console.log(c);
```

Explanation :

1. "Hello" and "World" exist separately in memory.
2. When concatenated, a new string "Hello World" is created in memory.
3. The variable c now references the new memory block.
4. The originals (a and b) remain unchanged.

Two ways to create strings:

Method	Example	Type of value
String literal (primitive)	let s1 = "hello";	Primitive string

String object (using new)	Let s2 = new String("hello");	String object (wrapper)
---------------------------	-------------------------------	-------------------------

Example :

```
let s2 = new String("hello");  
console.log(s2);
```

Explanation :

1. Creates a String object, not a primitive.
2. This object wraps a primitive string value inside it.
3. Stored in the heap as a full-fledged object.
4. It has methods and properties available because it's an object wrapper around the string.

Difference between String Literal and String Object

```
let s1 = "john";  
let s2 = new String("john");  
  
console.log(s1 == s2); // true → value is same (type coercion)  
console.log(s1 === s2); // false → different types (primitive vs object)
```

Step	let s1 = "hello"	let s2 = new String("hello")
Storage	Stored as a primitive value in the string area (stack or interned)	Allocates a new object on the heap
Value	"hello"	Object containing "hello"
Reference	Variable directly holds the value	Variable holds a reference to the object
Mutability	Immutable	The wrapper object is mutable (but the inner string value isn't)

String Methods:

Method	Description	Example
length	Returns the length of the string	"Hello".length → 5
toUpperCase()	Converts all characters to uppercase	"hello".toUpperCase() → "HELLO"
toLowerCase()	Converts all characters to lowercase	"HELLO".toLowerCase() → "hello"
trim()	Removes whitespace from both ends	" hi ".trim() → "hi"
includes()	Checks if substring exists	"hello".includes("he") → true
indexOf()	Finds first index of substring	"hello".indexOf("l") → 2

lastIndexOf()	Finds last index of substring	"hello".lastIndexOf("l") → 3
slice()	Extracts a portion of the string	"hello".slice(1, 4) → "ell"
substring()	Similar to slice (no negative index)	"hello".substring(1, 4) → "ell"
replace()	Replaces the first match with new text	"hello".replace("h", "y") → "yello"
replaceAll()	Replaces all matches with new text	"hi hi".replaceAll("hi", "hey") → "hey hey"
split()	Splits string into an array	"a,b,c".split(",") → ["a", "b", "c"]
concat()	Joins two or more strings	"Hello".concat(" World") → "Hello World"

startsWith()	Checks if string begins with given text	"hello".startsWith("he") → true
endsWith()	Checks if string ends with given text	"hello".endsWith("lo") → true

Example :

```
let str = "  JavaScript is Awesome!  ";

// 1. trim() - Remove extra spaces
let trimmed = str.trim();
console.log("After trim():", trimmed);

// 2. toUpperCase() - Convert to uppercase
let upper = trimmed.toUpperCase();
console.log("After toUpperCase():", upper);

// 3. toLowerCase() - Convert to lowercase
let lower = trimmed.toLowerCase();
console.log("After toLowerCase():", lower);

// 4. includes() - Check if a word exists
console.log("includes('Awesome'):", trimmed.includes("Awesome"));
console.log("includes('Python'):", trimmed.includes("Python"));

// 5. indexOf() - Find first index of a word
console.log("indexOf('is'):", trimmed.indexOf("is"));

// 6. slice() - Extract a part of the string
let sliced = trimmed.slice(0, 10);
console.log("After slice(0, 10):", sliced);

// 7. replace() - Replace a word
let replaced = trimmed.replace("Awesome", "Powerful");
console.log("After replace():", replaced);
```

```
// 8. split() - Split into array
let words = trimmed.split(" ");
console.log("After split():", words);

// 9. concat() - Join two strings
let newSentence = trimmed.concat(" Let's learn it!");
console.log("After concat():", newSentence);

// 10. startsWith() & endsWith() - Check string beginning and end
console.log("startsWith('Java'):", trimmed.startsWith("Java"));
console.log("endsWith('Awesome!'):", trimmed.endsWith("Awesome!"));
```

Arrays :

An array in JavaScript is a collection of elements (values or variables) stored in a single variable.

Declaring an Array :

Method	Example	Explanation
Array Literal (Preferred)	let arr = [10, 20, 30];	Most common and simplest way to create arrays.
Using new Keyword	let arr = new Array(10, 20, 30);	Creates an array object, but less often used.
Empty Array	let arr = [];	Used when you want to start with an empty array and add elements later.

1. Array Literal :

This is the most common and simplest way, using square brackets [].


```
const fruits = ["Apple", "Banana", "Cherry"]; // Array of strings
const mixed = [10, "Hello", true, { key: "value" }]; // Array of mixed types
const emptyArray = []; // An empty array

console.log(fruits);
```

2. Array Constructor :

Using the new `Array()` constructor. This is generally less preferred due to potential confusion when passing a single number.

```
const ages = new Array(25, 30, 35); // Creates [25, 30, 35]
const tenSlots = new Array(10); // Creates an array with 10 empty slots

console.log(ages)
console.log(tenSlots)
```

Array Characteristics :

1. They are resizable (their size can change).
2. They can hold items of different data types (e.g., numbers, strings, objects, or even other arrays).
3. They are zero-indexed, meaning the first element is at index 0.

Accessing and Modifying Elements

```
const colors = ["Red", "Green", "Blue"];
console.log(colors[0]); // Output: "Red"
console.log(colors[2]); // Output: "Blue"
console.log(colors[3]); // Output: undefined (index out of bounds)

const numbers = [1, 2, 3];
numbers[1] = 5; // Changes the second element
console.log(numbers); // Output: [1, 5, 3]

const items = ["A", "B", "C", "D"];
console.log(items.length); // Output: 4
```

Array Methods :

Method	Purpose	Example
<code>push()</code>	Adds a new element at the end	<code>arr.push(5)</code>
<code>pop()</code>	Removes the last element	<code>arr.pop()</code>
<code>shift()</code>	Removes the first element	<code>arr.shift()</code>
<code>unshift()</code>	Adds element at the beginning	<code>arr.unshift(5)</code>
<code>concat()</code>	Combines arrays	<code>[1,2].concat([3,4])</code>
<code>join()</code>	Converts array to a single string	<code>["a","b"].join("-")</code>
<code>slice()</code>	Returns selected elements	<code>arr.slice(1,3)</code>
<code>splice()</code>	Adds/removes elements	<code>arr.splice(2,1,"new")</code>
<code>indexOf()</code>	Finds index of a value	<code>arr.indexOf("apple")</code>
<code>includes()</code>	Checks if a value exists	<code>arr.includes("banana")</code>
<code>reverse()</code>	Reverses element order	<code>arr.reverse()</code>
<code>sort()</code>	Sorts elements alphabetically	<code>arr.sort()</code>
<code>forEach()</code>	Iterates through array	<code>arr.forEach(x => console.log(x))</code>
<code>map()</code>	Creates new array from callback	<code>arr.map(x => x * 2)</code>
<code>filter()</code>	Keeps elements that meet condition	<code>arr.filter(x => x > 10)</code>

reduce()	Reduces array to one value	arr.reduce((a,b)=>a+b)
----------	----------------------------	------------------------

Example :

```
// 1. Create an initial array
let numbers = [10, 20, 30, 40, 50];
console.log("Initial Array:", numbers);

// 2. push() - Add element at end
numbers.push(60);
console.log("After push(60):", numbers);

// 3. unshift() - Add element at beginning
numbers.unshift(0);
console.log("After unshift(0):", numbers);

// 4. pop() - Remove last element
numbers.pop();
console.log("After pop():", numbers);

// 5. shift() - Remove first element
numbers.shift();
console.log("After shift():", numbers);

// 6. concat() - Combine with another array
let extraNumbers = [70, 80, 90];
let combined = numbers.concat(extraNumbers);
console.log("After concat():", combined);

// 7. slice() - Copy part of the array (non-destructive)
let sliced = combined.slice(2, 5);
console.log("After slice(2,5):", sliced);

// 8. splice() - Remove and insert elements (destructive)
combined.splice(3, 2, 100, 110);
console.log("After splice(3,2,100,110):", combined);

// 9. includes() - Check if value exists
console.log("Has 40?", combined.includes(40));

// 10. indexOf() - Find index of a value
console.log("Index of 100:", combined.indexOf(100));
```

```
// 11. sort() - Sort elements (numeric ascending)
combined.sort((a, b) => a - b);
console.log("After sort():", combined);

// 12. reverse() - Reverse the order
combined.reverse();
console.log("After reverse():", combined);

// 13. map() - Create new array by doubling values
let doubled = combined.map(num => num * 2);
console.log("After map(x => x*2):", doubled);

// 14. filter() - Keep only values > 50
let filtered = combined.filter(num => num > 50);
console.log("After filter(num > 50):", filtered);

// 15. reduce() - Sum of all elements
let total = combined.reduce((sum, val) => sum + val, 0);
console.log("Sum of all elements (reduce):", total);

// 16. forEach() - Print all elements
console.log("Final Combined Array (using forEach):");
combined.forEach(num => console.log(num));
```

Boolean :

A Boolean in JavaScript is a primitive data type that holds only two possible values — true or false. It plays a key role in controlling the program's flow, forming the foundation of logical operations and conditional statements in JavaScript.

Creating Boolean Values

1. Literal Values

```
let isLoggedIn = true;
let hasError = false;

if (isLoggedIn) {
  console.log("Welcome back!");
}
```

2. Using the Boolean() Function

```
console.log(Boolean(10));    // true
console.log(Boolean(0));     // false
console.log(Boolean("Hello")); // true
console.log(Boolean(""));    // false
console.log(Boolean(null));  // false
```

3. Using Comparison Operators

```
let a = 10;
let b = 5;

console.log(a > b);    // true
console.log(a === b);  // false
```

Truthy and Falsy Values :

JavaScript has a concept called Type Coercion, where values that are not explicitly true or false are implicitly evaluated as such in a Boolean context (like an if statement).

1. Falsy Values:

These are the only six values that evaluate to false when converted to a Boolean:

- a. false (the Boolean literal)
- b. 0 (the number zero)
- c. "" or ' ' (an empty string)
- d. null
- e. undefined
- f. NaN (Not a Number)

2. Truthy Values:

Every other value in JavaScript is considered truthy and evaluates to true. This includes:

- a. Any non-zero number (e.g., 1, -5, 0.5)
- b. Any non-empty string (e.g., "0", "false", " ")

- c. Arrays (even empty ones: [])
- d. Objects (even empty ones: {})
- e. Functions

```
let userName = ""; // Falsy value
let userAge = 25; // Truthy value

if (userName) {
  // This block will NOT execute because "" is falsy.
  console.log("Username is set.");
}

if (userAge) {
  // This block WILL execute because 25 is truthy.
  console.log("Age is valid.");
}
```